

C++26 std::execution

2025/10/17
C++MIX #16

はじめに

誰？

twitter(X) @yohhoy / hatena id:yohhoy



何を？

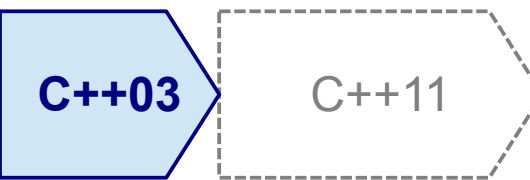
C++26 非同期・並行実行フレームワーク **std::execution** の紹介

どうして？

cpprefjpサイトにリファレンスをたくさん書いた (WIP)

これから日本語圏の情報やライブラリ拡張が増えてほしい

C++の非同期・並行処理



C++03

C++11

言語拡張/ライブラリ仕様

OpenMP, Open MPI, etc.

プラットフォーム依存ライブラリ

POSIXスレッド(Pthreads), Windows API, etc.

サードパーティ製ライブラリ

Boost.Thread, QThread(Qt), etc.

C++の非同期・並行処理

C++11

プログラミング言語仕様として
「スレッド」を正式サポート
(a.k.a. C++メモリモデル)

<thread>

標準スレッド `std::thread`

<mutex>

排他制御 `std::mutex`

RAIIロック `std::unique_lock`

<condition_variable>

条件変数 `std::condition_variable`

<future>

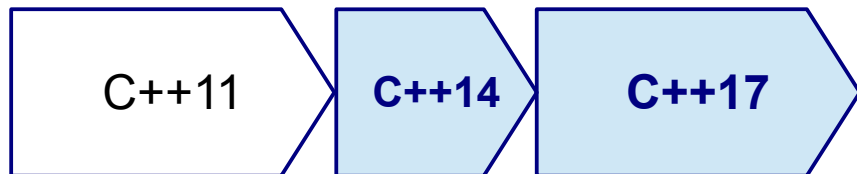
タスク実行 `std::async`

Promise/Future機構

<atomic>

アトミック変数 `std::atomic`

C++の非同期・並行処理



<algorithm> (既存)

並列実行アルゴリズム(Parallel STL)

<execution>

並列実行ポリシー `std::execution::par`

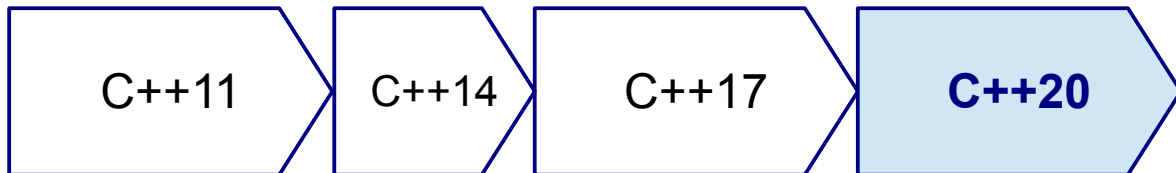
<mutex> (既存)

Readers/Writer排他制御 `std::shared_mutex`

Readerロック `std::shared_lock`

アルゴリズムに対する
スレッド並列実行許可ポリシー

C++の非同期・並行処理



<thread> (既存)

自動Joinスレッド `std::jthread`

<stop_token>

停止トークン `std::stop_token`

C++標準による汎用の
非同期キャンセル機構

<semaphore>

計数セマフォ

<latch>

ラッチ同期 `std::latch`

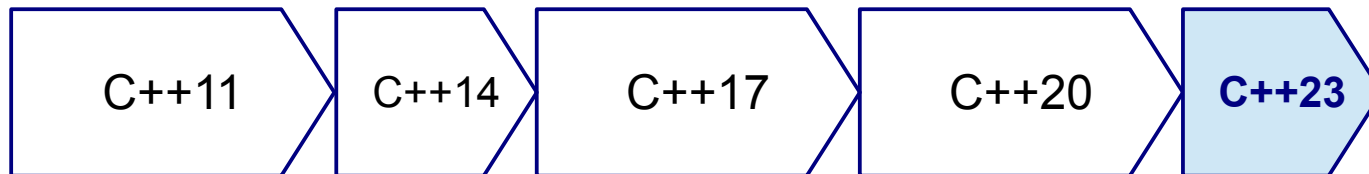
<barrier>

バリア同期 `std::barrier`

<atomic> (既存)

アトミック参照 `std::atomic_ref`

C++の非同期・並行処理



(仕様調整のみ)

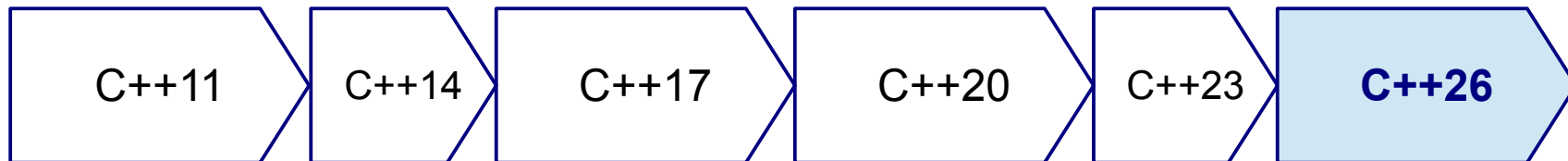
P2588R3 barrier's phase completion guarantees

<https://wg21.link/p2588r3>

P0943R6 Support C atomics in C++

<https://wg21.link/p0943r6>

C++の非同期・並行処理



<stop_token> (既存)

停止トークン追加

<execution> (既存)

実行制御ライブラリ

C++標準による汎用の
非同期・並行実行フレームワーク

<rcu>

Read Copy Update機構

<hazard_pointer>

Hazard Pointer

<simd>

データ並列ライブラリ

実行制御ライブラリ

(提案文書P2300R10引用)

This paper proposes a self-contained design for a Standard C++ framework for managing asynchronous execution on generic execution resources. It is based on the ideas in A Unified Executors Proposal for C++ and its companion papers.

(機械翻訳)

本論文は、汎用的な実行リソース上で非同期実行を管理するための、標準C++向け自己完結型フレームワークの設計を提案する。A Unified Executors Proposal for C++ およびその関連論文のアイデアに基づいている。

<https://wg21.link/p2300r10> P2300R10 std::execution

実行制御ライブラリ

(提案文書P2300R10引用)

This pa
manag
the ide
(機械
本論文
完結型
びその

非同期・並行実行の**統合モデル**として
汎用的かつ拡張可能な**フレームワーク**を
C++標準ライブラリとして定義する

or
on

自己
およ

実行制御ライブラリ

提案文書	タイトル	Word数
P2300R10	std::execution	47,500
P3325R5	A Utility for Creating Execution Environments	2,100
P3396R1	std::execution wording fixes	3,000
P2079R10	Parallel scheduler	9,400
P3481R4	std::execution::bulk() issues	4,000
P3149R11	async_scope - Creating scopes for non-sequential concurrency	16,300
P3557R3	High-Quality Sender Diagnostics with Constexpr Exceptions	8,600
P3552R3	Add a Coroutine Task Type	11,400
P3570R2	optional variants in sender/receiver	3,100
P3284R4	write_env and unstoppable Sender Adaptors	1,300

2025年9月現在、採択済み主要提案文書のサイズ（Microsoft Wordで計数）

実行制御ライブラリ

提案文書	タイトル	Word数
P2300R10	std::execution	47,500
P3325R5	A Utility for Creating Execution	2,100
P3396R1	std::execution work	3,000
P2079R10	Parallel	9,400
P3481R4	std::execution	4,000
P3149R11	asynchronous	16,300
P3557R3	High-Quantum Exception	8,600
P3552R3	Add a C	11,400
P3570R2	optional variants sender/receiver	3,100
P3284R4	write_env and unstoppable Sender Adaptors	1,300

Total:
100,000+ Words

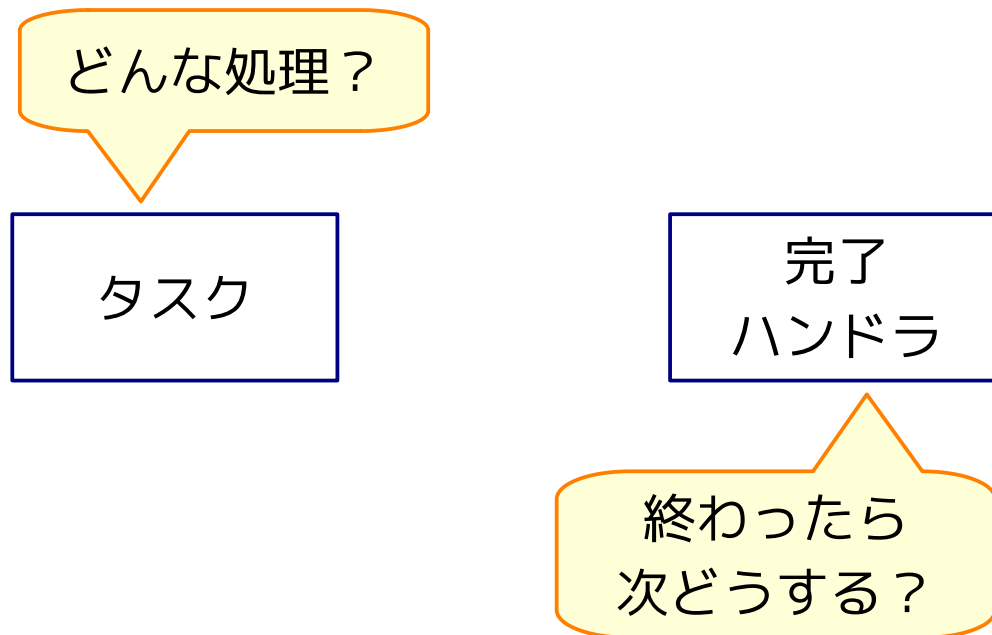
~_(\ツ)_/~

2025年9月現在、採択済み主要提案文書のサイズ（Microsoft Wordで計数）

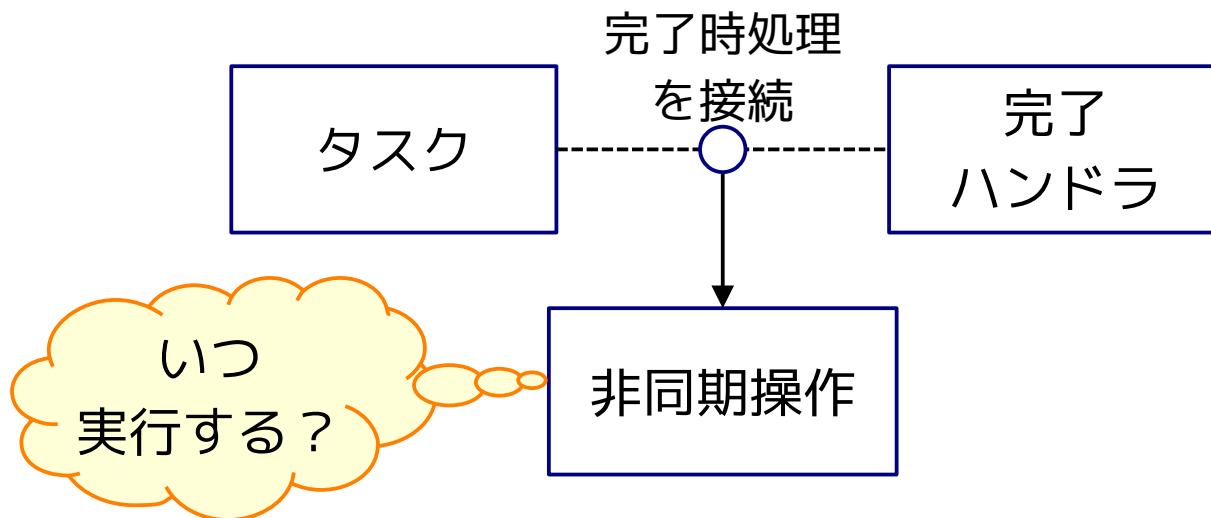
C++26 実行制御ライブラリ

(Execution Control Library)

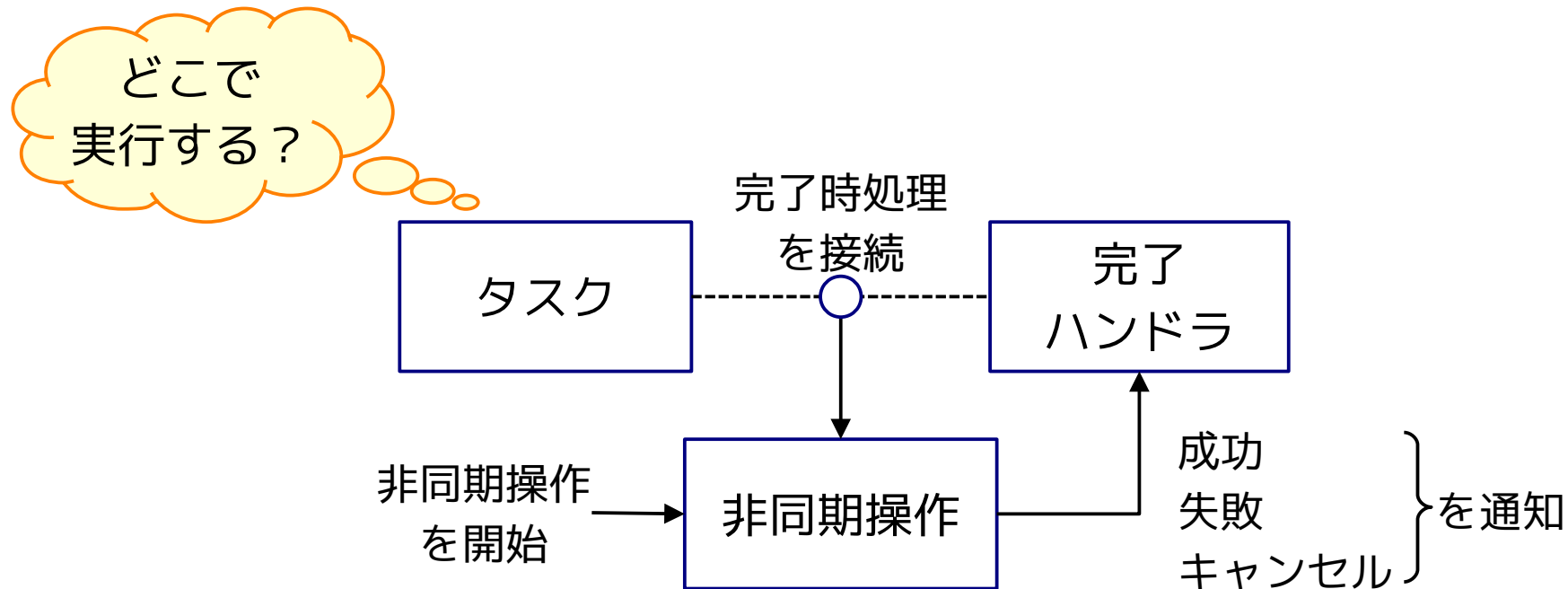
非同期処理のモデル化



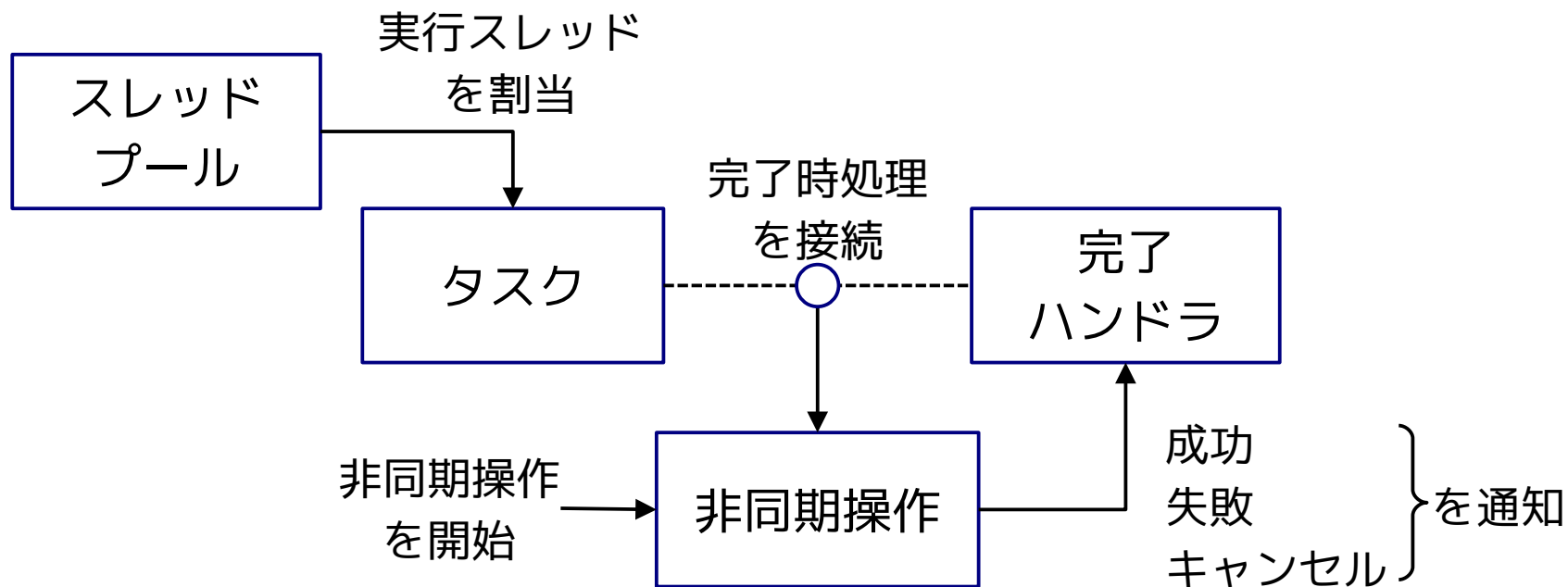
非同期処理のモデル化



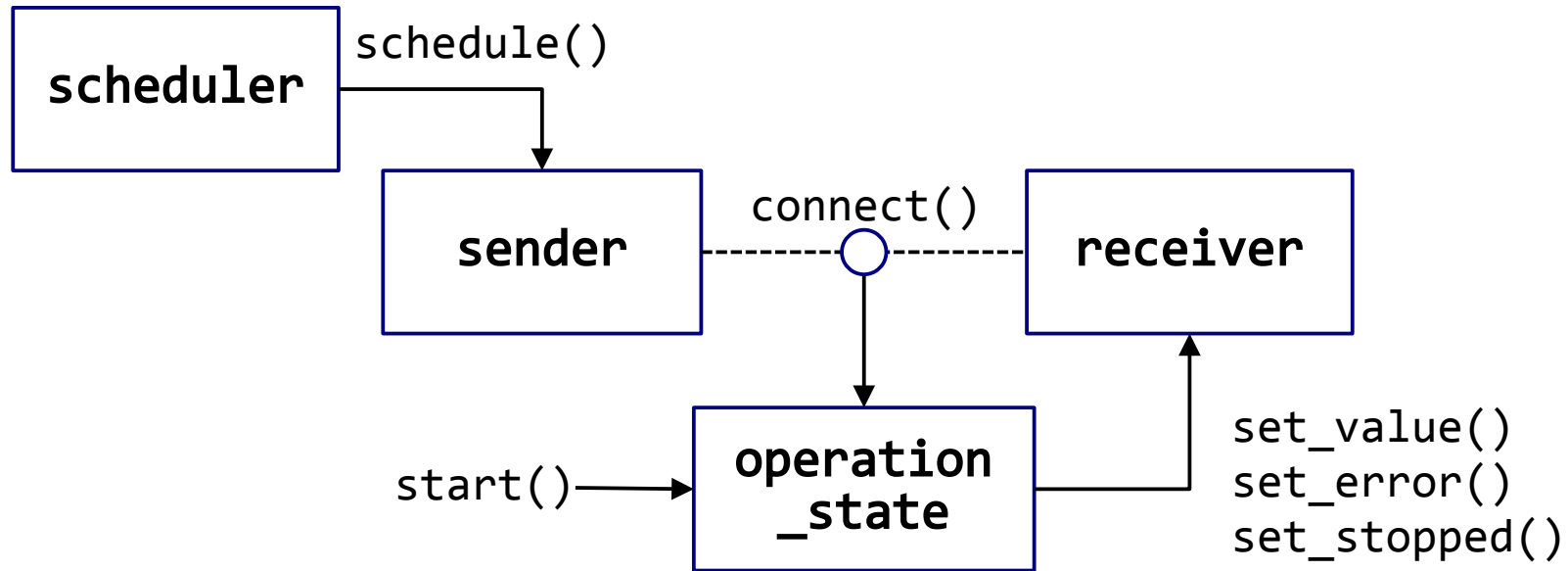
非同期処理のモデル化



非同期処理のモデル化

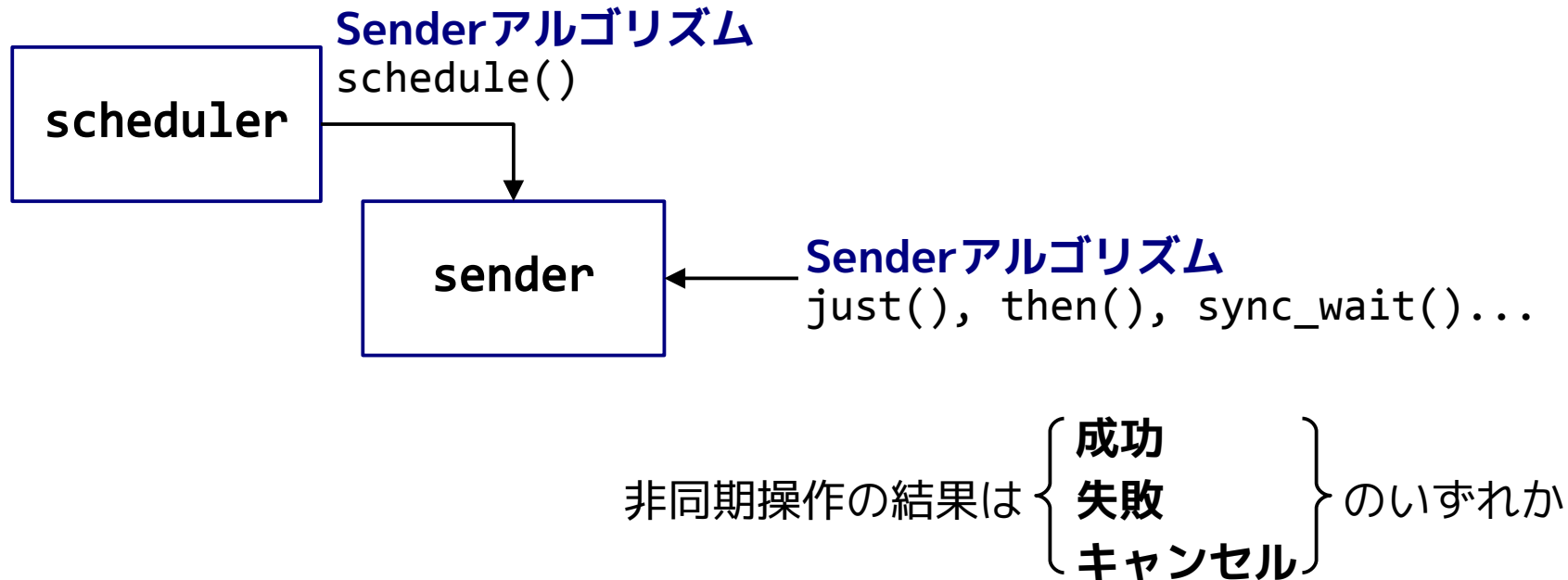


Senders/Receiversモデル



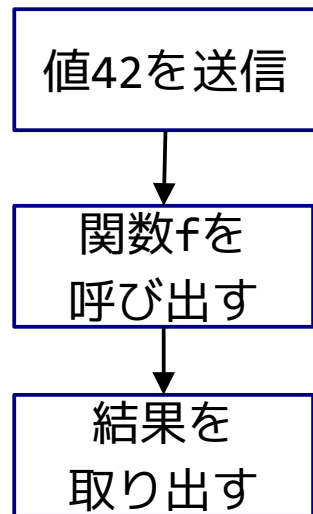
Senders/Receiversモデル

～ 利用ユーザ視点 ～



Sender

非同期・並行実行されるタスクを **Sender** として定義する
Sender型は **std::execution::sender** コンセプトを満たす
複数のSenderを接続してタスクチェーンを構成する

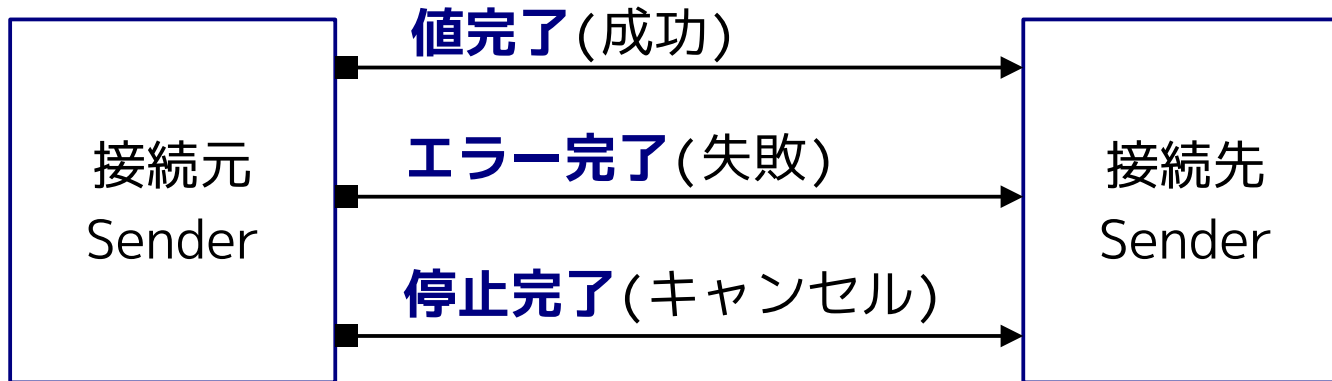


対応する逐次処理

```
auto n = 42;  
auto r = f(n);  
return r;
```

完了シグネチャ

Sender間には3種類の通信チャネルで接続され、**いずれか1つ**が利用される
取りうる値/エラー/停止完了の型情報を**完了シグネチャ**の集合で表現
接続元 $\subseteq$ 接続先の包含関係にないときは接続失敗する（コンパイルエラー）



完了シグネチャ

	Senders/Receivers 完了シグネチャ	C++関数 関数シグネチャ
成功	<code>set_value_t(Vs...)</code>	戻り値型 (多値返却: <code>tuple<Vs...></code>)
失敗	<code>set_error_t(Err)</code>	<code>noexcept</code> 指定 (例外の静的型Errは表現不可)
キャンセル	<code>set_stopped_t()</code>	N/A

`set_{value,error,stopped}_t` は名前空間 `std::execution` で定義される未規定の型

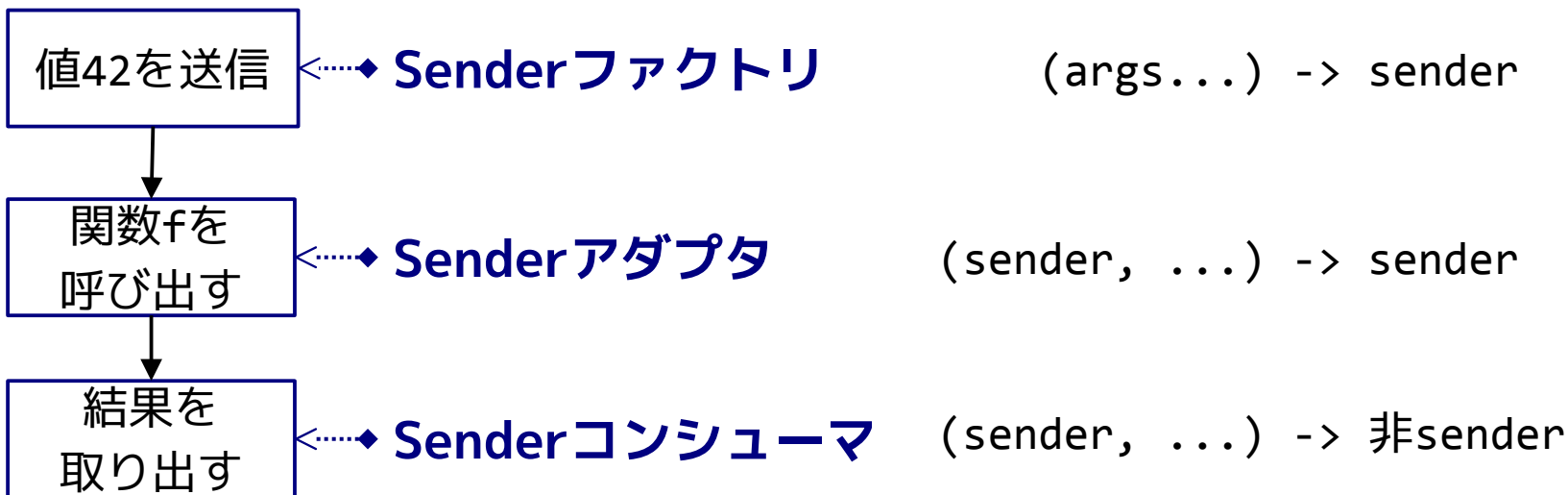
完了シグネチャ集合

Sender	関数
<code>completion_signatures< set_values_t(int) ></code>	<code>int() noexcept</code>
<code>completion_signatures< set_values_t(float, float), set_error_t(exception_ptr) ></code>	<code>tuple<float, float>()</code>
<code>completion_signatures< set_values_t(int), set_values_t(float, float), set_error_t(error_code), set_stopped_t(), ></code>	<code>optional< expected< variant< tuple<int>, tuple<float, float>>, error_code>>() noexcept</code>

完了シグネチャ集合を `std::execution::completion_signatures<Sigs...>` で表現

Senderアルゴリズム

Senderによるタスクチェインを生成／接続(connect)する
タスクチェインを開始(start)し、完了待機してから結果取得する



Senderアルゴリズム

Senderファクトリ

just
just_error
just_stopped
schedule
read_env

Senderアダプタ

write_env
unstoppable
start_on
continues_on
on
schedule_from
then
upon_error
upon_stopped
let_value
let_error
let_stopped
bulk
bulk_chunked
bulk_unchunked
when_all
when_all_variant
into_variant
stopped_as_optional
stopped_as_error
associate
spawn_future
affine_on

Senderコンシューマ

sync_wait
sync_wait_with_variant
spawn

Senderアルゴリズムは名前空間 `std::execution` で定義される

`sync_wait(_with_varinat)` の2種のみ名前空間 `std::this_thread` で定義される

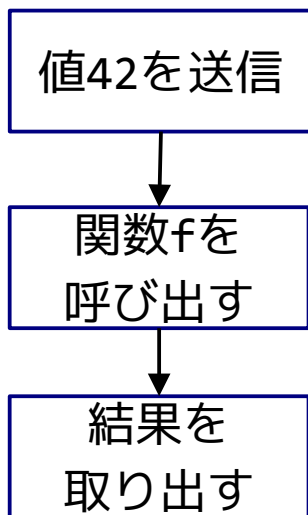
Slideware™

```
#include <execution>
#include <stop_token>
#include <thread>
#include <vector>
using namespace std;
namespace ex = std::execution;

int f(int); // 関数 int -> int
```

タスクチェーン

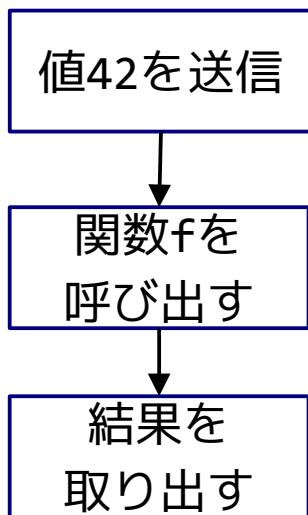
Senderによるタスクチェーンを生成&接続(connect)する
タスクチェーンを開始(start)&完了待機し、結果を取り出す



```
// 値42を送信
ex::sender auto snd0 = ex::just(42);
// 関数fを呼び出す
ex::sender auto snd1 = ex::then(snd0, f);
// タスクチェーンを開始し結果を取り出す
auto result = this_thread::sync_wait(snd1);
```

パイプライン記法

Senderアダプタは**パイプライン記法**(a|b)でも接続可能※
(Senderコンシューマはパイプライン記法をサポートしない)



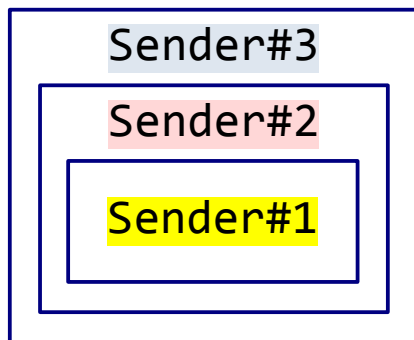
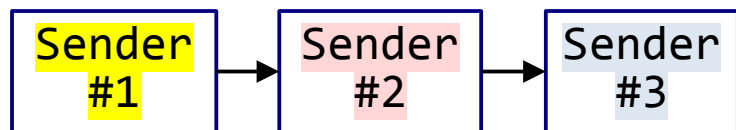
```
// 値42を送信+関数fを呼び出す(パイプライン記法)
ex::sender auto sndr =
    ex::just(42) | ex::then(f);

// タスクチェーンを開始し結果を取り出す
auto result = this_thread::sync_wait(sndr);
```

※ API設計上、意図的にパイプライン記法をサポートしないSenderアダプタもある

パイプライン記法

パイプライン記法は糖衣構文(Syntax sugar)として導入
実際のタスクチェーンSenderは入れ子構造として構築される

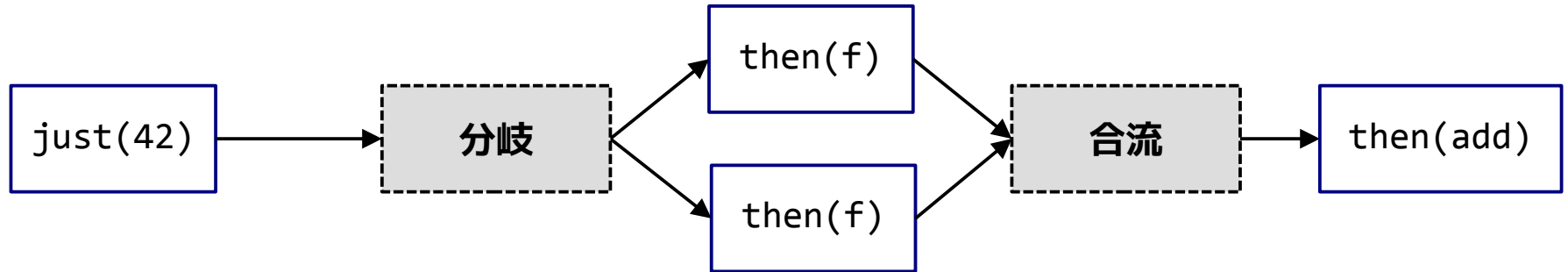


```
// タスクチェーン構築(パイプライン記法)  
ex::sender auto sndr =  
    sender1 | sender2 | sender3;
```

```
// タスクチェーン構築(関数呼び出し)  
ex::sender auto sndr =  
    sender3(sender2(sender1));
```

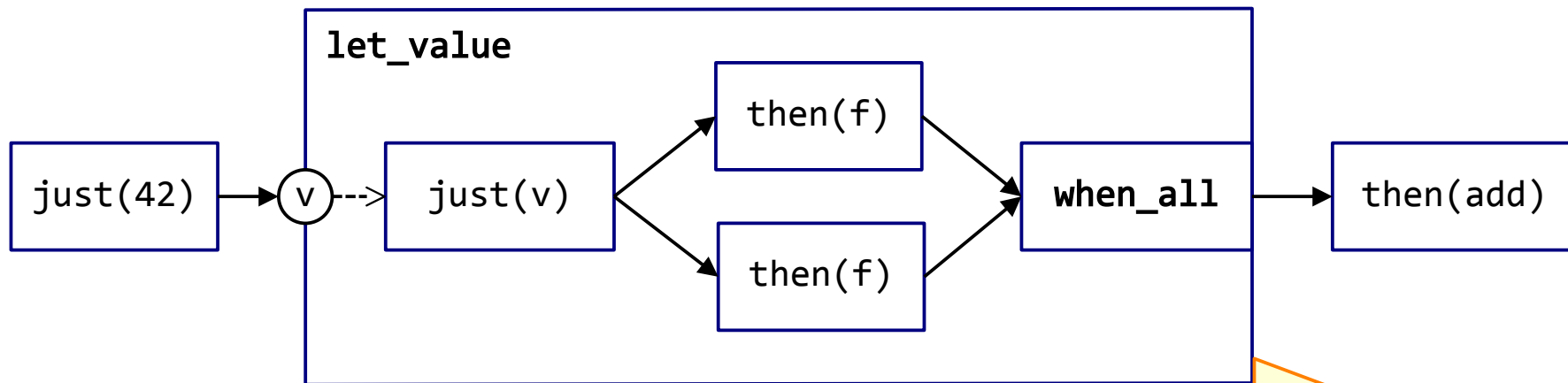
分岐と合流

非同期処理の分岐と合流を表現するタスクチェーン



分岐と合流

非同期処理の分岐と合流を表現するタスクチェーン



let_valueからの返却Senderが
後続thenと接続される
(Monadic bind操作)

分岐と合流

非同期処理の分岐と合流を表現するタスクチェーン

```
int add(int, int);  
// タスクチェーンを定義  
ex::sender auto sndr =  
    ex::just(42)  
    | ex::let_value([](int v) {  
        ex::sender auto just_v = ex::just(v);  
        // 受信値vを送信するSenderを複製(分岐)  
        return ex::when_all(  
            just_v | ex::then(f),  
            just_v | ex::then(f),  
        );  
    })  
    | ex::then(add);  
// タスクチェーンを開始し結果を取り出す  
auto result = this_thread::sync_wait(sndr);
```


this_thread::sync_wait

呼び出しスレッドをブロックし、結果取得するSenderコンシューマ

	戻り値
成功	<code>optional<tuple<Vs...>></code>
失敗	エラー値Errを例外送出
キャンセル	<code>nullopt</code> (<code>optional</code> 型の無効値)

単一の値型Vでも
`tuple<V>`となる

this_thread::sync_wait

呼び出しスレッドをブロックし、結果取得するSenderコンシューマ

```
ex::sender auto sndr = /* タスクチェイン */;
try {
    auto result = this_thread::sync_wait(sndr);
    // 戻り値result := optional<tuple<Vs...>>型
    if (result) {
        auto vals = result.value();
        // tuple<Vs...> := 値完了(成功)
    } else {
        // nullopt := 停止完了(キャンセル)
    }
} catch (...) {
    // 例外送出 := エラー完了(失敗)
}
```

Senderアルゴリズム

Senderファクトリ

just
just_error
just_stopped
schedule
read_env

Senderアダプタ

write_env
unstoppable
start_on
continues_on
on
schedule_from
then
upon_error
upon_stopped
let_value
let_error
let_stopped

bulk
bulk_chunked
bulk_unchunked
when_all
when_all_variant
into_variant
stopped_as_optional
stopped_as_error
associate
spawn_future
affine_on

Senderコンシューマ

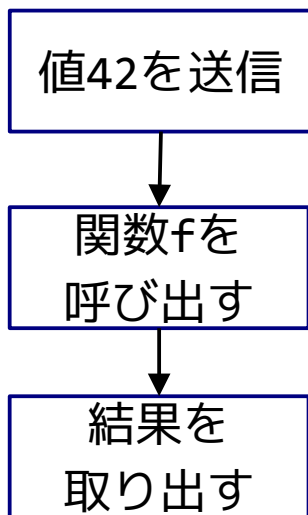
sync_wait
sync_wait_with_variant
spawn



遅延開始

Senderによるタスクチェーンは明示的な開始操作が必要

sync_waitアルゴリズム = **遅延開始(lazy start)**と完了待機を同時に行う

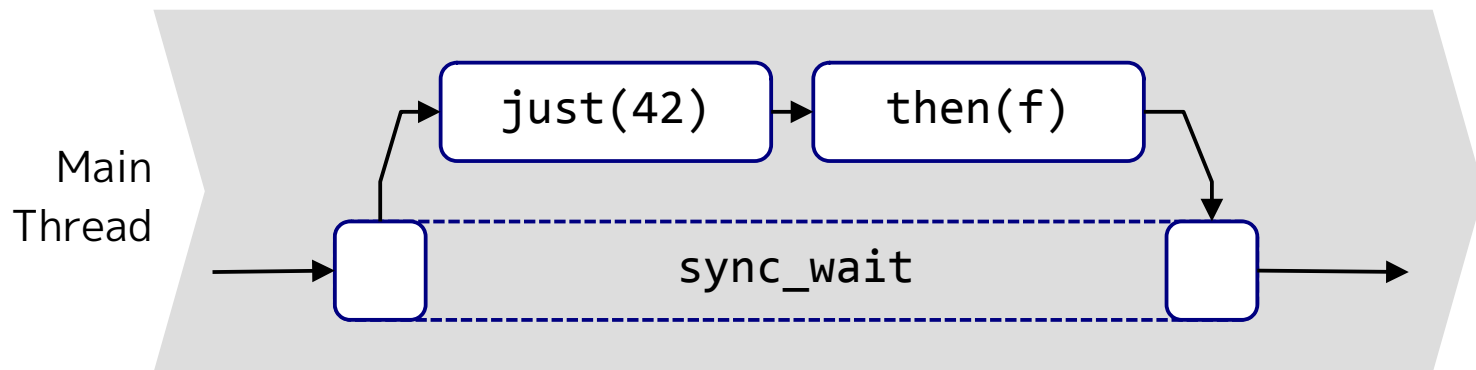


```
// タスクチェーンを定義
ex::sender auto sndr = ex::just(42) | ex::then(f);

// タスクチェーンを開始し結果を取り出す
auto result = this_thread::sync_wait(sndr);
```

自スレッド上でタスクを実行

遲延開始



```
ex::sender auto sndr = ex::just(42) | ex::then(f);  
this_thread::sync_wait(sndr);
```

Scheduler

タスクを実行するスレッドを **Scheduler** で指定する
システムスレッドプールや自前スレッド上での実行をサポート※

```
// システムスレッドプールのSchedulerを取得
ex::scheduler auto sch = ex::get_parallel_scheduler();
// タスクチェーンを定義
ex::sender auto sndr = ex::just(42) | ex::then(f);
// スレッドプール上でタスクを開始(自スレッドはブロック)
auto result = this_thread::sync_wait(ex::on(sch, sndr));
```

システムスレッドプールの
詳細仕様は処理系定義

※ CPUスレッド以外も想定した汎用的な仕様となっている。NVIDIAによるGPU/CUDA実装あり。

Scheduler

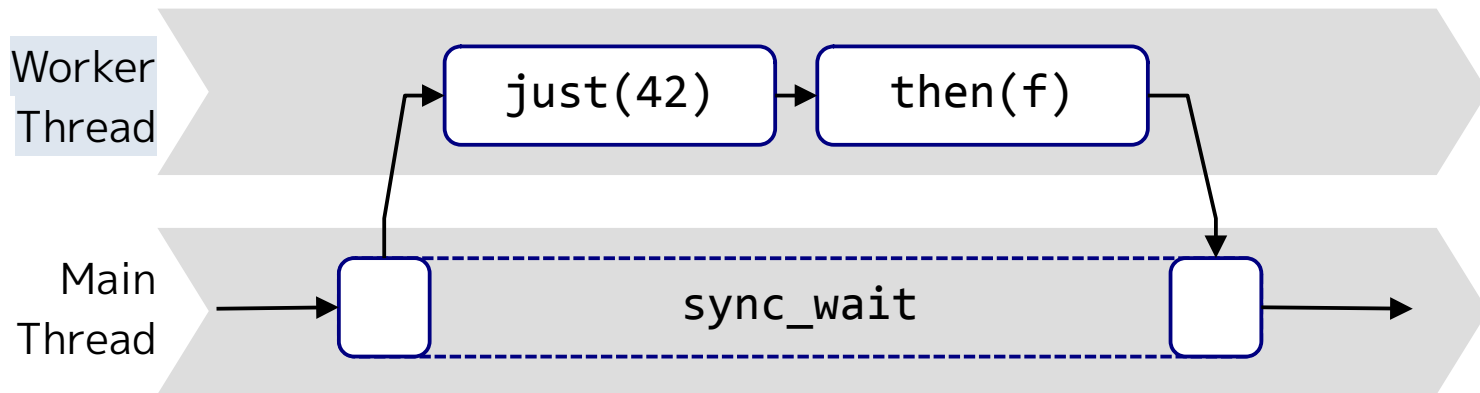
タスクを実行するスレッドを **Scheduler** で指定する
システムスレッドプールや自前スレッド上での実行をサポート

```
// タスクキューのSchedulerを取得
ex::run_loop loop;
ex::scheduler auto sch = loop.get_scheduler();
// ワークスレッド上でタスクキューを実行
jthread worker{[&]{ loop.run(); }};

// タスクチェーンを定義
ex::sender auto sndr = ex::just(42) | ex::then(f);
// ワークスレッド上でタスクを開始(自スレッドはブロック)
auto result = this_thread::sync_wait(ex::on(sch, sndr));
// タスクキューへの終了通知
loop.finish();
```

スレッド間の値／エラー授受は
ライブラリ内で安全に行われる

Scheduler



```
ex::scheduler auto sch = /* Scheduler取得 */  
ex::sender auto sndr = ex::just(42) | ex::then(f);  
this_thread::sync_wait(ex::on(sch, sndr));
```


早期開始

非同期スコープ+spawnアルゴリズムによる**早期開始(eager start)**

```
// スレッドプール上で実行されるタスクチェーンを構築(戻り値なし)
ex::scheduler auto sch = ex::get_parallel_scheduler();
ex::sender auto sndr =
    ex::schedule(sch) | ex::then([]{ /* 処理A */ });

// 非同期スコープを定義
ex::counting_scope scope;
// スレッドプール上でタスクを早期開始(非同期スコープと関連付け)
ex::spawn(move(sndr), scope.get_token());
/* 自スレッドでの処理B */

// スレッドプール上でのタスク完了を待機
this_thread::sync_wait(scope.join());
```

早期開始

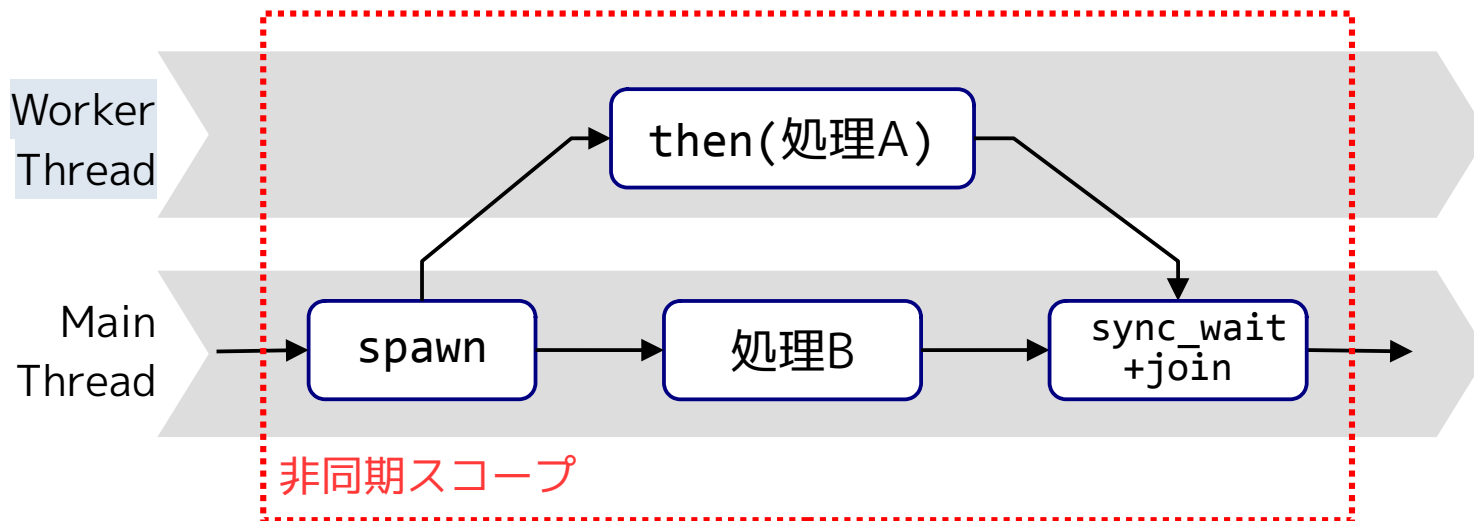
非同期スコープ+spawn_futureアルゴリズムによる**早期開始(eager start)**

```
// スレッドプール上で実行されるタスクチェーンを構築(戻り値あり)
ex::scheduler auto sch = ex::get_parallel_scheduler();
ex::sender auto sndr =
    ex::schedule(sch) | ex::then([]{ return /* 処理A */; });

// 非同期スコープを定義
ex::counting_scope scope;
// スレッドプール上でタスクを早期開始(非同期スコープと関連付け)
ex::sender auto future =
    ex::spawn_future(move(sndr), scope.get_token());
/* 自スレッドでの処理B */

// スレッドプール上でのタスク完了待機
auto result = this_thread::sync_wait(
    ex::when_all(move(future), scope.join()));
```

早期開始



```
ex::scheduler auto sch = /* Scheduler取得 */  
ex::counting_scope scope;  
ex::spawn(ex::schedule(sch) | ex::then(/*処理A*/), scope.get_token());  
/* 処理B */  
this_thread::sync_wait(scope.join());
```

並行実行

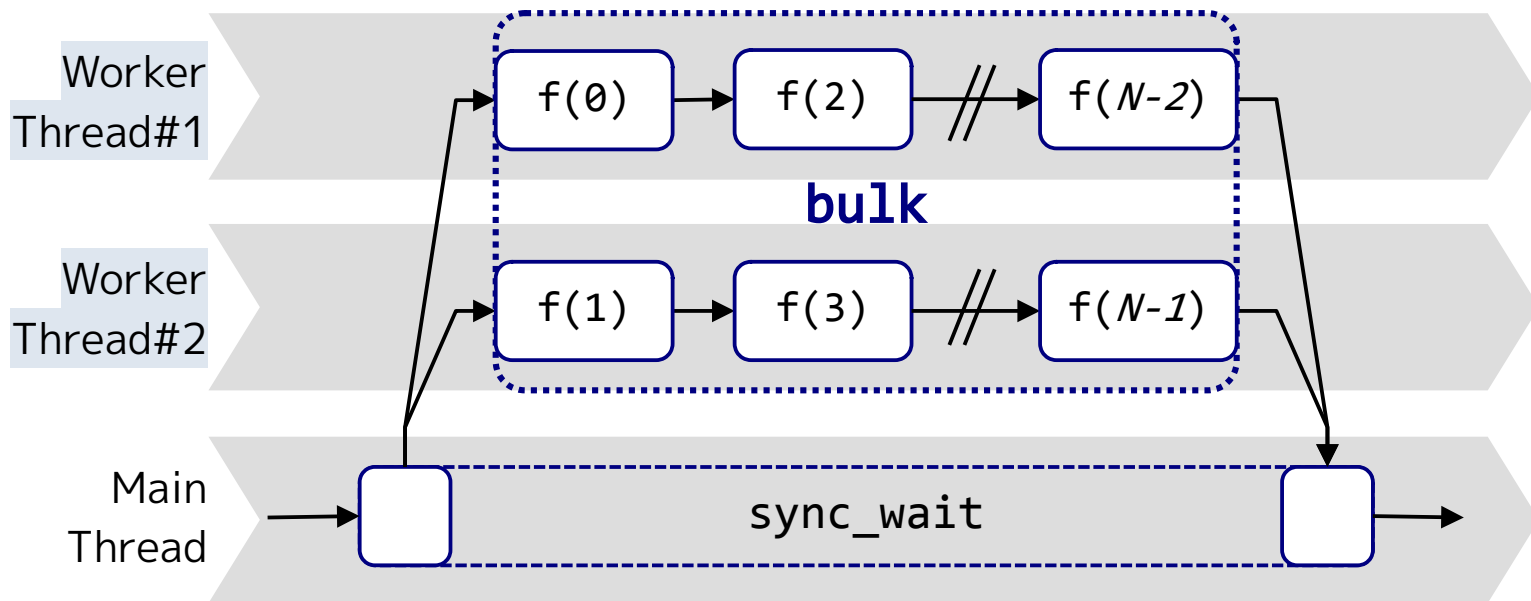
システムスレッドプール+bulkアルゴリズムによるタスク並行実行

```
// データセット
vector<int> data = /*...*/;
const size_t N = data.size();

// システムスレッドプールで実行されるタスクチェインを定義
ex::scheduler auto sch = ex::get_parallel_scheduler();
ex::sender auto sndr =
    ex::schedule(sch)
    | ex::bulk(
        ex::par, N,
        [&](size_t idx){ f(data[idx]); });
// スレッドプール上でタスク開始し完了待機
this_thread::sync_wait(sndr);
```

スレッドプールのスレッド群で
idx=[0,N)範囲が並列実行される

並行実行



```
ex::sender auto sndr =  
    ex::schedule(ex::get_parallel_scheduler())  
    | ex::bulk(std::par, N, [](auto idx){ f(idx); });  
this_thread::sync_wait(move(sndr));
```

並行実行

システムスレッドプール+bulkアルゴリズムによるタスク並行実行

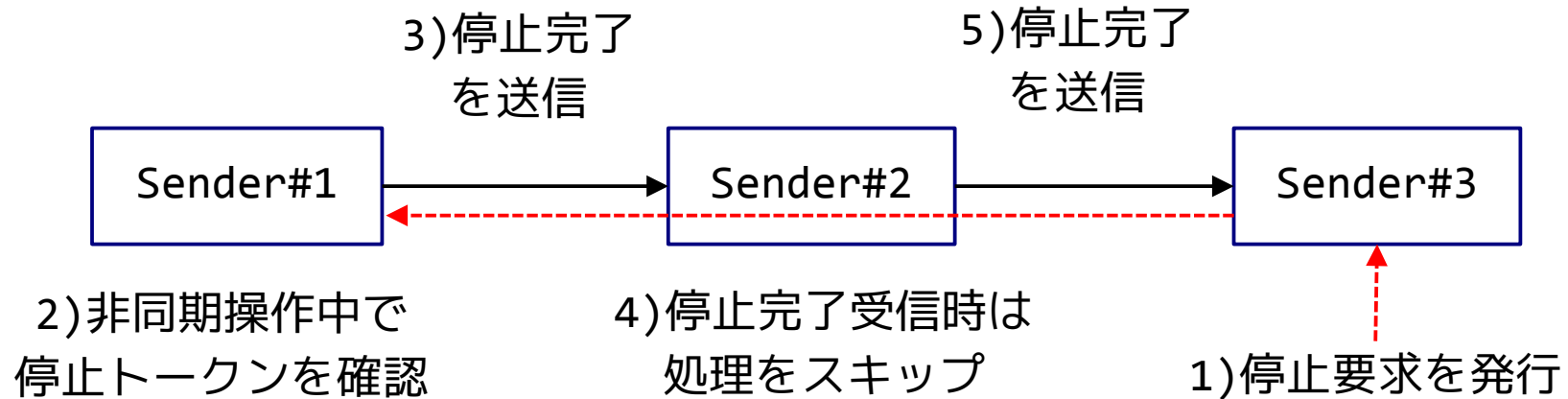
```
// データセット
vector<int> data = /*...*/;
const size_t N = data.size();

// システムスレッドプールで実行されるタスクチェーンを定義
ex::scheduler auto sch = ex::get_parallel_scheduler();
ex::sender auto sndr =
    ex::schedule(sch)
    | ex::bulk(
        ex::seq, N,
        [&](size_t idx){ f(data[idx]); });
// スレッドプール上でタスク開始し完了待機
this_thread::sync_wait(sndr);
```

単一スレッド上での
逐次実行に変更(par→seq)

非同期キャンセル

停止トークンを用いた**協調的キャンセル動作**による非同期キャンセル
(Sender#1が停止トークンに応答 / Sender#3で停止要求を発行可能とする)

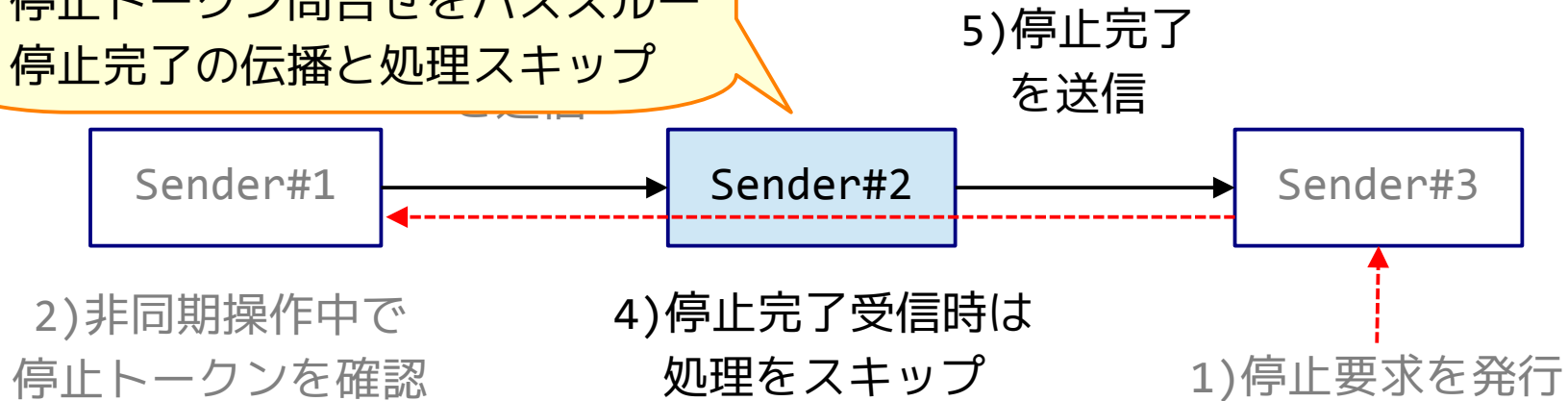


非同期キャンセル

停止トークンを用いた**協調的キャンセル動作**による非同期キャンセル

標準Senderアダプタの基本動作

- 停止トークン問合せをパススルー
- 停止完了の伝播と処理スキップ



非同期キャンセル

停止トークンを用いた**協調的キャンセル動作**による非同期キャンセル

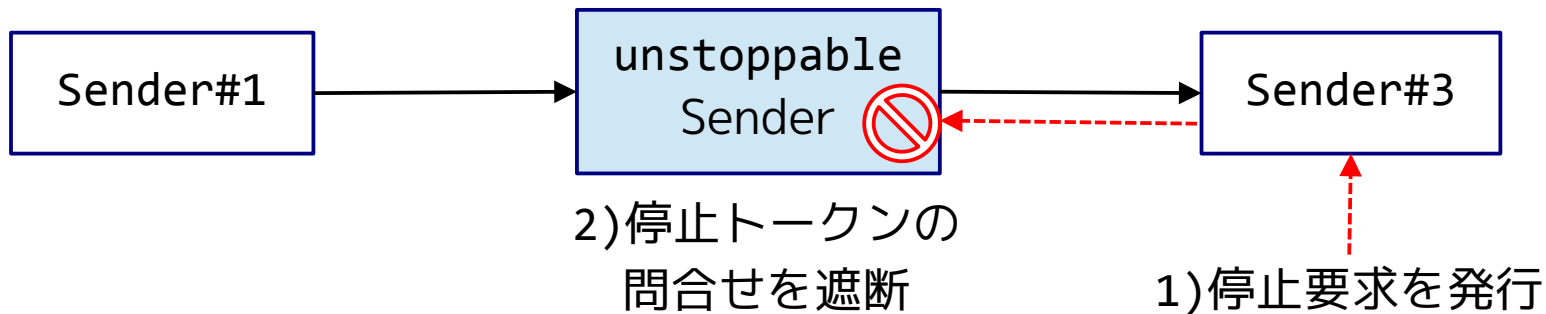
```
// タスクチェインを定義
struct MySender; // 非同期キャンセル対応Sender型(int値を送信)
ex::sender auto snd0 = MySender{} | ex::then(f);
// 停止要求に対応したSenderと接続
stop_source stop_src;
ex::sender auto snd1 = ex::write_env(move(snd0),
    ex::prop(get_stop_token, stop_src.get_token()));

// 停止要求を発行(本来は別スレッドから)
stop_src.request_stop();
// タスクチェイン開始=停止完了となり関数fは呼び出されない
this_thread::sync_wait(move(snd1)); // nulloptを返す
```

非同期キャンセル

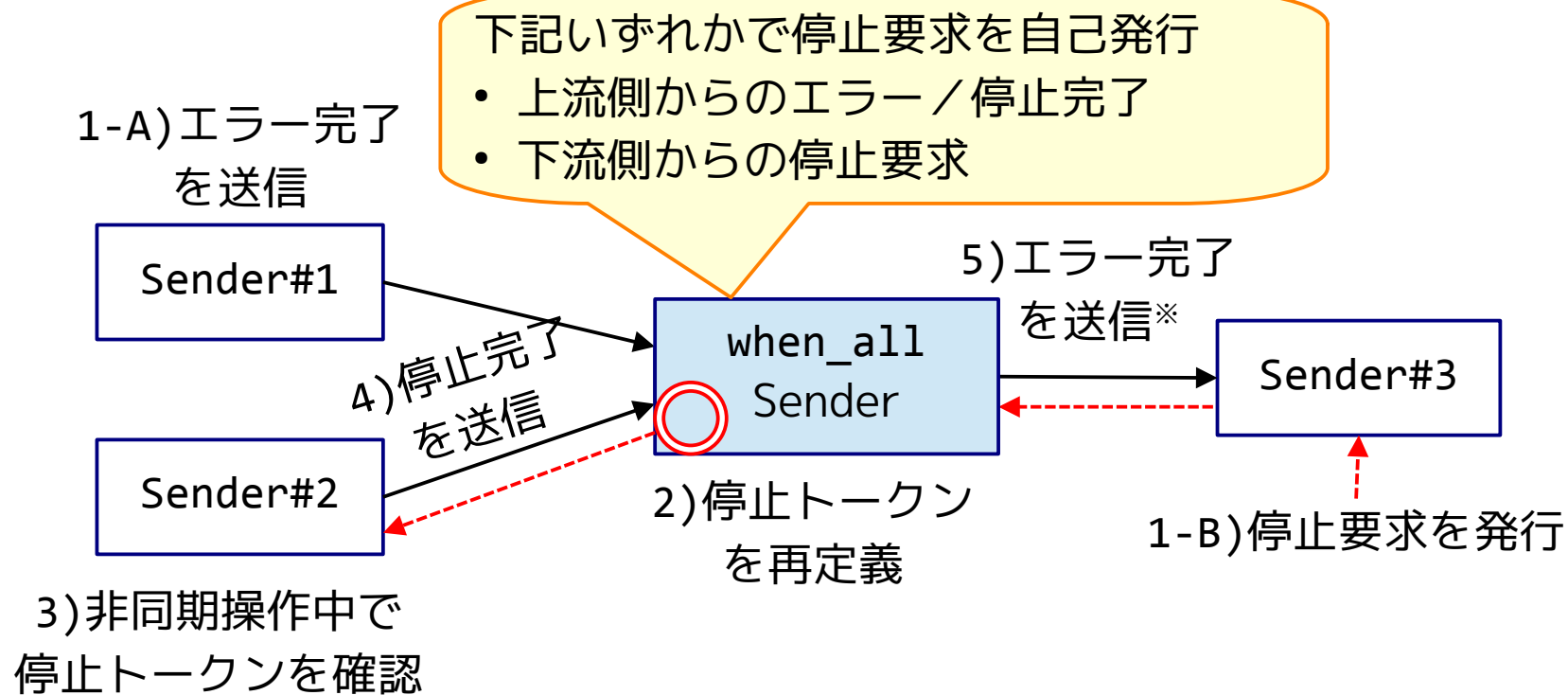
unstoppableアルゴリズム

- 停止トークン問合せを遮断※
- 停止完了は伝播させる



※ 上流側からの問合せを下流側に転送せず、代わりにnever_stop_tokenを返す。

非同期キャンセル



※ いずれかでエラー→初回エラーで完了／いずれかで停止→停止完了／全て成功→値完了

Senderアルゴリズム

Senderファクトリ

`just`
`just_error`
`just_stopped`
`schedule`
`read_env`

Senderアダプタ

`write_env`
`unstoppable`
`start_on`
`continues_on`
`on`
`schedule_from`
`then`
`upon_error`
`upon_stopped`
`let_value`
`let_error`
`let_stopped`

`bulk`
`bulk_chunked`
`bulk_unchunked`
`when_all`
`when_all_variant`
`into_variant`
`stopped_as_optional`
`stopped_as_error`
`associate`
`spawn_future`
`affine_on`

Senderコンシューマ

`sync_wait`
`sync_wait_with_variant`
`spawn`



Senderアルゴリズム × ユーザ定義Sender

標準Senderアルゴリズムはドメイン非依存の汎用部品

実用的にはユーザ定義Senderと標準Senderアルゴリズムを組み合わせる

- ネットワークI/O
- ファイルI/O
- GUIイベントループ
- タイマー／スケジューリング処理
- etc.

ユーザ定義Sender

仕様書

- int型の値 val を1つ引数にとる
- 停止要求が行われていたら、停止完了(キャンセル)
- val が0以上のとき、val で値完了(成功)
- そうでなければ、例外でエラー完了(失敗)

ユーザ定義Sender

```
template<typename Rcvr>
struct OpState {
    using operation_state_concept
        = ex::operation_state_t;

    void start() & noexcept {
        auto st = get_stop_token(ex::get_env(rcvr_));
        if (st.stop_requested()) {
            ex::set_stopped(move(rcvr_));
            return;
        }
        if (0 <= val_) {
            ex::set_value(move(rcvr_), val_);
        } else {
            ex::set_error(move(rcvr_),
                          invalid_argument{"NG"});
        }
    }

    Rcvr rcvr_;
    int val_;
};
// (cont.)
```

```
// (cont.)
struct MySender {
    using sender_concept = ex::sender_t;

    template<typename Self>
    static constexpr auto get_completion_signatures() {
        return ex::completion_signatures<
            ex::set_value_t(int),
            ex::set_error_t(exception_ptr),
            ex::set_stopped_t()
        >{};
    }

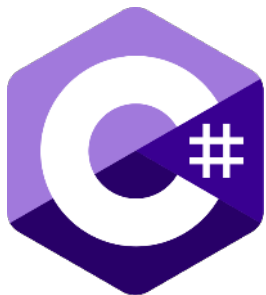
    template<typename Rcvr>
    auto connect(Rcvr rcvr) {
        return OpState{move(rcvr), val_};
    }

    int val_;
};

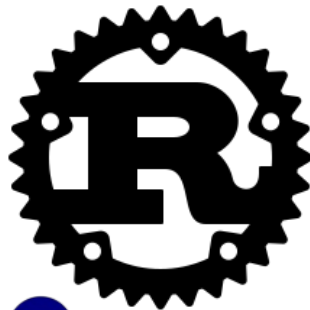
// ex::sender auto sndr = MySender{n};
```

非同期処理 == コルーチン ?

C#



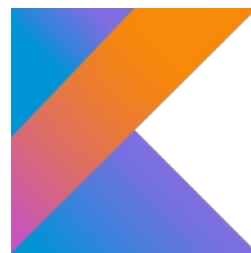
Rust



Python



Lua

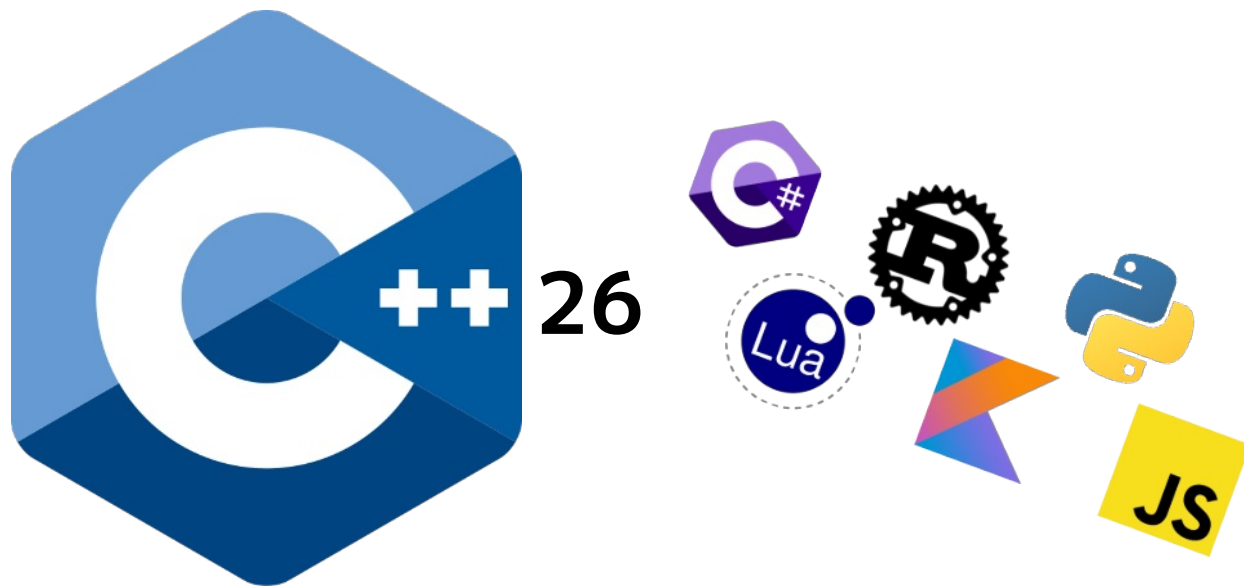


Kotlin



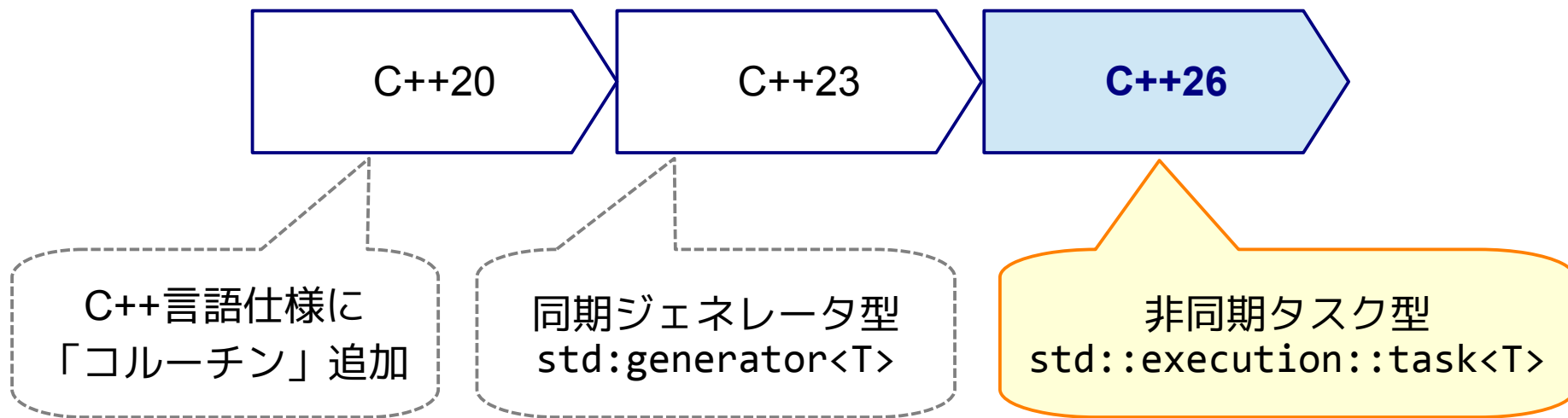
JavaScript

Senders/Receivers c コルーチン



execution::task<T>

C++標準の非同期タスク・コルーチンの戻り値型
Senders/Receiversモデルとの相互運用を前提に設計



コルーチン相互運用

任意のコルーチンAwaitable型==Senderとして扱える※

```
// int値を返すユーザ定義コルーチン
ex::task<int> coro1() { co_return 6; }

ex::task<int> coro2() {
    int n = co_await coro1(); // Awaitable型
    co_return n * 7;
}

// コルーチン実行+関数fを呼び出すタスクチェーンを定義
ex::sender auto sndr = coro2() | ex::then(f);
// タスクチェーンを開始し結果を取り出す
auto result = this_thread::sync_wait(move(sndr));
```

※ 実行制御ライブラリと無関係なユーザ定義コルーチンAwaitable型もSenderとして扱える

コルーチン相互運用

コルーチン内ではSenderを**Await式**(co_await)で待機可能※

```
// int値を返すユーザ定義コルーチン
ex::task<int> coro() {
    // タスクチェーンを定義(戻り値int型)
    ex::sender auto sndr = ex::just(42) | ex::then(f);

    // タスク(Sender)の実行結果をAwait式で取得
    int result = co_await sndr;

    co_return result;
}
```

※ コルーチンPromise型がunhandled_stopped関数を実装していること
また単一の値完了シグネチャをもつSenderであること（エラー完了と停止完了もサポート）

ユーザ定義Sender (再掲)

```
template<typename Rcvr>
struct OpState {
    using operation_state_concept
        = ex::operation_state_t;

    void start() & noexcept {
        auto st = get_stop_token(ex::get_env(rcvr_));
        if (st.stop_requested()) {
            ex::set_stopped(move(rcvr_));
            return;
        }
        if (0 <= val_) {
            ex::set_value(move(rcvr_), val_);
        } else {
            ex::set_error(move(rcvr_),
                          invalid_argument{"NG"});
        }
    }

    Rcvr rcvr_;
    int val_;
};
// (cont.)
```

```
// (cont.)
struct MySender {
    using sender_concept = ex::sender_t;

    template<typename Self>
    static constexpr auto get_completion_signatures() {
        return ex::completion_signatures<
            ex::set_value_t(int),
            ex::set_error_t(exception_ptr),
            ex::set_stopped_t()
        >{};
    }

    template<typename Rcvr>
    auto connect(Rcvr rcvr) {
        return OpState{move(rcvr), val_};
    }

    int val_;
};
```

ユーザ定義コルーチン

```
ex::task<int> MyTask(int val)
{
    // 停止トークンを確認
    auto st = co_await ex::read_env(get_stop_token);
    if (st.stop_requested()) {
        // キャンセル(停止完了)
        co_await ex::just_stopped();
    }
    if (0 <= val) {
        // 成功(値完了)
        co_return val;
    } else {
        // 失敗(エラー完了)
        throw invalid_argument{"NG"};
    }
}
```

ユーザ定義コルーチン

```
ex::task<int> MyTask(int val)
{
    // 停止トークンを確認
    auto st = co_await ex::read_env(get_stop_token);
    if (st.stop_requested()) {
        // キャンセル(停止完了)
        co_await ex::just_stopped();
    }
    if (0 <= val) {
        // 成功(値完了)
        co_return val;
    } else {
        // 失敗(エラー完了)
        throw invalid_argument{"NG"};
    }
}
```

仕様書

- int型の値 `val` を1つ引数にとる
- 停止要求が行われていたら、`停止完了`(キャンセル)
- `val` が0以上のとき、`val` で値完了(成功)
- そうでなければ、`例外でエラー完了`(失敗)

まとめ

C++26実行制御ライブラリ

C++標準の **非同期・並行実行モデル** を定義するフレームワーク
あらゆる非同期・並行実行タスクを **Sender** として表現
基本的なビルディングブロック **Senderアルゴリズム** を多数提供
標準ライブラリで **システムスレッドプール** を定義
ユーザ定義の非同期処理を **コルーチン** として記述可能



Visit “cpprefjp” website !

<https://cpprefjp.github.io/reference/execution/execution.html>

CAUTION

**SPEC. UNDER
CONSTRUCTION**



WG21 mailing2025-10

P3826R0 Defer Sender Algorithm Customization to C++29

標準Senderアルゴリズムのカスタマイズ機構は不完全で機能しない
C++26ではカスタマイズ機構を削除しC++29に向けて再検討すべき

この提案では...

- 1) C++標準から実行制御ライブラリを全て削除
- 2) カスタマイズ可能なSenderアルゴリズムのみを削除
- 3) 現行のカスタマイズ機構を削除して簡略仕様を導入**
- 4) C++26リリース後にDR(Defect Report)で修正する



<https://wg21.link/p3826r0>



WG21 mailing2025-10

P3845R0 Make `std::execution`'s monadic operations naming scheme consistent

標準ライブラリその他Monadic操作と一貫性のある名称を用いるべき

比較対象：`std::optional<T>`, `std::expected<T, E>`

この提案では...

<code>ex::then</code>	→ <code>ex::transform</code>
<code>ex::let_value</code>	→ <code>ex::and_then</code>
<code>ex::let_error</code>	→ <code>ex::or_else_error</code>
<code>ex::let_stopped</code>	→ <code>ex::or_else_stopped</code>

<https://wg21.link/p3845r0>

[参考] 関連リンク

ライブラリ実装 (P2300ベース)

<https://github.com/NVIDIA/stdexec>

<https://github.com/intel/cpp-baremetal-senders-and-receivers>

<https://github.com/bemanproject/execution>

Structured Concurrency

<https://ericniebler.com/2020/11/08/structured-concurrency/>

Asynchronous Stacks and Scopes

<https://ericniebler.com/2021/08/29/asynchronous-stacks-and-scopes/>

What are Senders Good For, Anyway?

<https://ericniebler.com/2024/02/04/what-are-senders-good-for-anyway/>

[参考] 関連リンク

WG21/P2300 std::execution

<https://zenn.dev/yohhoy/scraps/5c500ed9096792>

C++26 Executionライブラリ（基礎編／Senderアルゴリズム編／コルーチン相互運用編）

<https://yohhoy.hatenadiary.jp/entry/20250609/p1>

<https://yohhoy.hatenadiary.jp/entry/20250612/p1>

<https://yohhoy.hatenadiary.jp/entry/20250613/p1>

C++26 Executionライブラリ：Cancellable Sender

<https://yohhoy.hatenadiary.jp/entry/20250625/p1>

C++26 Executionライブラリ：Sender完了シグネチャ集合

<https://yohhoy.hatenadiary.jp/entry/20250823/p1>

C++26 Executionライブラリ：非同期スコープ

<https://yohhoy.hatenadiary.jp/entry/20251002/p1>

[参考] 関連リンク

Boost.勉強会#14 「C++時代の新しい並列for構文のご提案」

<https://yohhoy.hatenablog.jp/entry/2014/03/02/143031>

<https://www.docswell.com/s/yohhoy/M533M5-cppparfor>

C++MIX#5 「20分くらいでわかった気分になれるC++20コルーチン」

<https://yohhoy.hatenablog.jp/entry/2019/09/06/174712>

<https://www.docswell.com/s/yohhoy/L57EJK-cpp20coro>

C++MIX#13 「サンプルコードによるC++23ジェネレータの紹介」

<https://yohhoy.hatenablog.jp/entry/2025/02/24/205436>

<https://www.docswell.com/s/yohhoy/KEXQPG-cpp23gen>

[予備] Senders/Receivers と コルーチン

	Senders/Receivers	C++ Coroutines
成功	複数の値完了シグネチャ	戻り値型は1種類に限定 (<code>variant<Ts...></code> で直和表現)
失敗	複数のエラー完了シグネチャ	例外機構に限定 (<code>expected<T,E></code> で代替可能)
キャンセル	標準化された停止完了	独自の通知機構が必要

[予備] コルーチンとScheduler制御

co_awaitによるコルーチン中断／再開前後でSchedulerを維持※

```
// ユーザ定義コルーチン
ex::task<void> coro(ex::scheduler auto sch2) {
    // 処理A @ Scheduler#1

    // 処理C @ Scheduler#2 の開始&完了待機
    co_await ex::schedule(sch2) | ex::then([]{ /*処理C*/ });

    // 処理B @ Scheduler#1
}
```

※ Scheduler affinityはスレッドの維持を意味しない (Ex. システムスレッドプール)

[予備] コルーチンとScheduler制御

コルーチン途中でのScheduler変更操作もサポート

```
// ユーザ定義コルーチン
ex::task<void> coro(ex::scheduler auto sch2) {
    // 処理A @ Scheduler#1

    // コルーチン内でScheduler切替
    co_await ex::change_coroutine_scheduler(sch2);

    // 処理B @ Scheduler#2
}
```